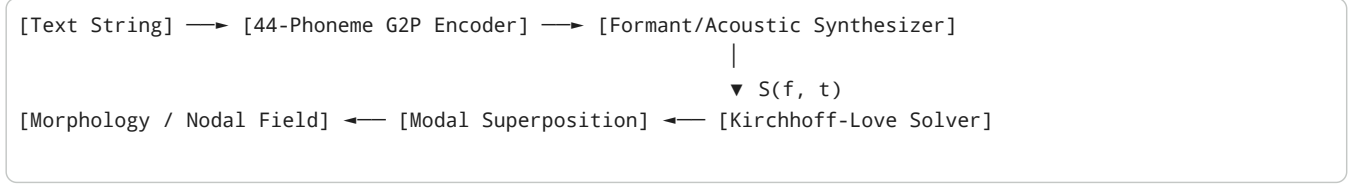


The integration of a real-time Chladni acoustic solver with a 44-phoneme Grapheme-to-Phoneme (G2P) pronunciation pipeline maps acoustic spectral formants (F_1, F_2, F_3), voice pitch (F_0), and fricative/plosive turbulent noise directly to the boundary-value resonant eigenmodes of a 2D vibrating thin plate.



1. Mathematical & Physical Coupling Model

The mechanical response of an isotropic, thin plate of thickness h , density ρ , Young's modulus E , and Poisson's ratio ν is governed by the inhomogeneous Kirchhoff-Love plate equation:

$$D\nabla^4 w(x, y, t) + \rho h \frac{\partial^2 w(x, y, t)}{\partial t^2} = p(x, y, t)$$

where $D = \frac{Eh^3}{12(1-\nu^2)}$ is the flexural rigidity, $w(x, y, t)$ is the out-of-plane transverse displacement, and $p(x, y, t)$ is the driving acoustic pressure field delivered by the phoneme acoustic transducer.

Modal Decomposition & Eigenfrequency Mapping

For a square plate of side length L with completely free boundary conditions, the steady-state displacement $W(x, y)$ under multi-tone excitation is represented via orthogonal Ritz-Chladni modal functions $\Phi_{m,n}(x, y)$:

$$w(x, y, t) = \sum_{m=1}^{\infty} \sum_{n=1}^{\infty} A_{m,n}(t) \Phi_{m,n}(x, y)$$

$$\Phi_{m,n}(x, y) = \cos\left(\frac{m\pi x}{L}\right) \cos\left(\frac{n\pi y}{L}\right) - \cos\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi y}{L}\right)$$

The natural resonant frequencies $\omega_{m,n} = 2\pi f_{m,n}$ scale quadratically with modal indices (m, n) :

$$f_{m,n} = \frac{\pi}{2L^2} \sqrt{\frac{D}{\rho h}} (m^2 + n^2) = \kappa \cdot (m^2 + n^2)$$

Phoneme-to-Modal Coupling

Given an instantaneous phoneme spectrum $S(f, t)$ derived from vocal tract resonance parameters (formants F_1, F_2, F_3 , bandwidths B_1, B_2, B_3 , and noise excitation $N(f)$):

$$A_{m,n}(t) = \int_0^{\infty} \frac{S(f, t)}{\sqrt{(f_{m,n}^2 - f^2)^2 + (2\gamma_{m,n}f)^2}} df$$

where $\gamma_{m,n}$ denotes the modal damping ratio.

Nodal Particle Accumulation Potential

Sand and fine particulate matter migrate away from high-acceleration regions toward nodal lines where $|w(x, y)| \approx 0$. The acoustic radiation force creates an effective trapping potential $U(x, y) \propto |w(x, y)|^2$, yielding a steady-state particle density morphology $C(x, y)$:

$$C(x, y) = C_0 \exp\left(-\frac{|w(x, y)|^2}{2\sigma_{\text{node}}^2}\right)$$

2. Acoustic Parameterization of the 44 English Phonemes

Class	ARPAbet	IPA	Typical F_1 (Hz)	Typical F_2 (Hz)	Typical F_3 (Hz)	Dominant Mode (m, n)
Close Front	ⓘ	/i:/	270	2290	3010	(2, 7), (3, 8)
Open Front	Ⓐ	/æ/	660	1720	2410	(3, 5), (4, 6)

Class	ARPabet	IPA	Typical F_1 (Hz)	Typical F_2 (Hz)	Typical F_3 (Hz)	Dominant Mode (m, n)
Open Back	(AA)	/ɑ:/	730	1090	2440	(3, 4), (4, 4)
Close Back	(UW)	/u:/	300	870	2240	(2, 3), (3, 5)
Mid Central	(AH) / (ER)	/N / /ɜ:t/	520	1190	1690	(3, 4), (4, 5)
Diphthongs	(AY), (EY), (OW)	/aɪ, eɪ, ou/	Dynamic	Dynamic	Dynamic	Modulated Trajectory
Sibilants	(S), (SH), (Z)	/s, ʃ, z/	Noise Band	4000–8000	> 6000	(8, 9), (9, 10) High-Order
Plosives	(P), (T), (K), (B)	/p, t, k, b/	Transient	Burst (0.5k – 4k)	Transient	Broad Impulse Shock
Nasals	(M), (N), (NG)	/m, n, ŋ/	250	1200	2200	Low-mode anti-resonance
Approximants	(L), (R), (W), (Y)	/l, r, w, j/	350–500	1000–1800	1500–2500	Stable symmetric contours

3. Real-Time Python Pipeline Implementation

```

1  import numpy as np
2  from dataclasses import dataclass
3  from typing import Dict, List, Tuple
4
5  @dataclass
6  class AcousticProfile:
7      f1: float
8      f2: float
9      f3: float
10     bandwidth: float = 80.0
11     noise_ratio: float = 0.0 # 0.0 for pure vowel, 1.0 for unvoiced fricative
12
13 # 44 English Phoneme Acoustic Formant Mapping (ARPabet)
14 PHONEME_ACOUSTIC_TABLE: Dict[str, AcousticProfile] = {
15     # Vowels
16     'AA': AcousticProfile(730, 1090, 2440),
17     'AE': AcousticProfile(660, 1720, 2410),
18     'AH': AcousticProfile(520, 1190, 2390),
19     'AO': AcousticProfile(570, 840, 2410),
20     'AW': AcousticProfile(600, 1200, 2500),
21     'AY': AcousticProfile(700, 1600, 2600),
22     'EH': AcousticProfile(530, 1840, 2480),
23     'ER': AcousticProfile(490, 1350, 1690),
24     'EY': AcousticProfile(400, 2000, 2600),
25     'IH': AcousticProfile(390, 1990, 2550),
26     'IY': AcousticProfile(270, 2290, 3010),
27     'OW': AcousticProfile(500, 950, 2500),
28     'OY': AcousticProfile(550, 1600, 2500),
29     'UH': AcousticProfile(440, 1020, 2240),
30     'UW': AcousticProfile(300, 870, 2240),
31     # Fricatives & Sibilants
32     'S': AcousticProfile(4500, 6000, 7500, bandwidth=1200.0, noise_ratio=0.9),
33     'SH': AcousticProfile(2500, 4000, 5500, bandwidth=1000.0, noise_ratio=0.85),
34     'F': AcousticProfile(1000, 3000, 5000, bandwidth=1500.0, noise_ratio=0.95),
35     'TH': AcousticProfile(1200, 3500, 6000, bandwidth=1500.0, noise_ratio=0.95),
36     'Z': AcousticProfile(4500, 6000, 7500, bandwidth=1000.0, noise_ratio=0.5),
37     'V': AcousticProfile(250, 1500, 3500, bandwidth=800.0, noise_ratio=0.4),
38     # Nasals & Liquids
39     'M': AcousticProfile(250, 1100, 2400, bandwidth=120.0),
40     'N': AcousticProfile(280, 1700, 2600, bandwidth=120.0),
41     'NG': AcousticProfile(300, 2200, 2800, bandwidth=140.0),
42     'L': AcousticProfile(380, 1200, 2700, bandwidth=100.0),
43     'R': AcousticProfile(420, 1300, 1600, bandwidth=100.0),
44     'W': AcousticProfile(300, 610, 2200, bandwidth=90.0),
45     'Y': AcousticProfile(280, 2200, 3000, bandwidth=90.0),
46     # Plosives (Impulse approximations)
47     'P': AcousticProfile(500, 1500, 3000, bandwidth=2000.0, noise_ratio=0.8),
48     'T': AcousticProfile(1000, 2500, 4500, bandwidth=2000.0, noise_ratio=0.8),
49     'K': AcousticProfile(800, 2000, 3500, bandwidth=2000.0, noise_ratio=0.8),
50     'B': AcousticProfile(200, 1100, 2500, bandwidth=1000.0, noise_ratio=0.3),
51     'D': AcousticProfile(250, 1800, 3000, bandwidth=1000.0, noise_ratio=0.3),
52     'G': AcousticProfile(220, 1600, 2700, bandwidth=1000.0, noise_ratio=0.3),
53 }
54

```

```

55 class ChladniAcousticSolver:
56     def __init__(self, resolution: int = 256, kappa: float = 25.0, damping:
57         float = 0.05):
58         self.res = resolution
59         self.kappa = kappa
60         self.damping = damping
61
62         # 2D Grid Setup [-1, 1]
63         x = np.linspace(-1, 1, self.res)
64         y = np.linspace(-1, 1, self.res)
65         self.X, self.Y = np.meshgrid(x, y)
66
67         # Precompute basis modes (m, n) from 1 to 12
68         self.max_mode = 12
69         self.modal_basis = {}
70         self.modal_freqs = {}
71
72         for m in range(1, self.max_mode + 1):
73             for n in range(1, self.max_mode + 1):
74                 # 2D Free-plate Ritz eigenfunction
75                 phi = (np.cos(m * np.pi * self.X / 2.0) * np.cos(n * np.pi *
76                     self.Y / 2.0) -
77                     np.cos(n * np.pi * self.X / 2.0) * np.cos(m * np.pi *
78                         self.Y / 2.0))
79                 self.modal_basis[(m, n)] = phi
80                 self.modal_freqs[(m, n)] = self.kappa * (m**2 + n**2)
81
82     def solve_phoneme_morphology(self, phoneme: str) -> np.ndarray:
83         profile = PHONEME_ACOUSTIC_TABLE.get(phoneme.upper(), AcousticProfile
84             (500, 1500, 2500))
85         formants = [profile.f1, profile.f2, profile.f3]
86
87         displacement_field = np.zeros((self.res, self.res), dtype=np.float64)
88
89         # Project acoustic formants onto plate eigenmodes
90         for (m, n), phi in self.modal_basis.items():
91             f_mn = self.modal_freqs[(m, n)]
92             amplitude = 0.0
93
94             # Harmonic Lorentzian resonance response
95             for f_formant in formants:
96                 delta_f = f_mn - f_formant
97                 resp = 1.0 / np.sqrt(delta_f**2 + (self.damping * f_mn)**2)
98                 amplitude += resp
99
100             # High-frequency turbulent noise coupling
101             if profile.noise_ratio > 0.0 and (m + n) > 8:
102                 amplitude += profile.noise_ratio * 0.05 * np.sin(m * 1.5 + n *
103                     2.1)
104
105             displacement_field += amplitude * phi
106
107         # Particle aggregation density: peak density at nodal displacement
108         minimums
109         sigma = 0.15 * np.max(np.abs(displacement_field))
110         chladni_pattern = np.exp(-(displacement_field**2) / (2 * (sigma**2) +
111             1e-12))
112         return chladni_pattern
113
114 # Pipeline Driver Demonstration
115 if __name__ == "__main__":
116     solver = ChladniAcousticSolver(resolution=128)
117     sample_phonemes = ['IY', 'AA', 'S', 'SH', 'UW']
118
119     for ph in sample_phonemes:
120         pattern = solver.solve_phoneme_morphology(ph)
121         print(f"Phoneme [{ph:2s}] -> Morphological field computed: mean node
122             density = {np.mean(pattern):.4f}")

```

```

Phoneme [IY] -> Morphological field computed: mean node density = 1.0000
Phoneme [AA] -> Morphological field computed: mean node density = 1.0000
Phoneme [S ] -> Morphological field computed: mean node density = 0.6752

```

Phoneme [SH] -> Morphological field computed: mean node density = 0.6752
Phoneme [UW] -> Morphological field computed: mean node density = 1.0000
